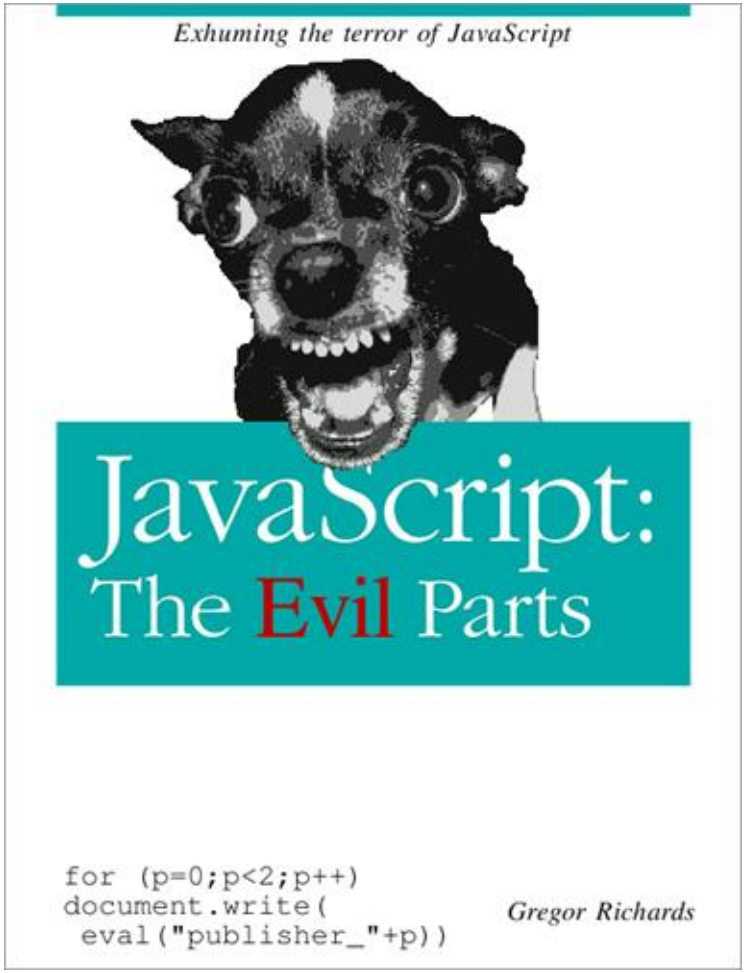
Client-side Dynamic Pages



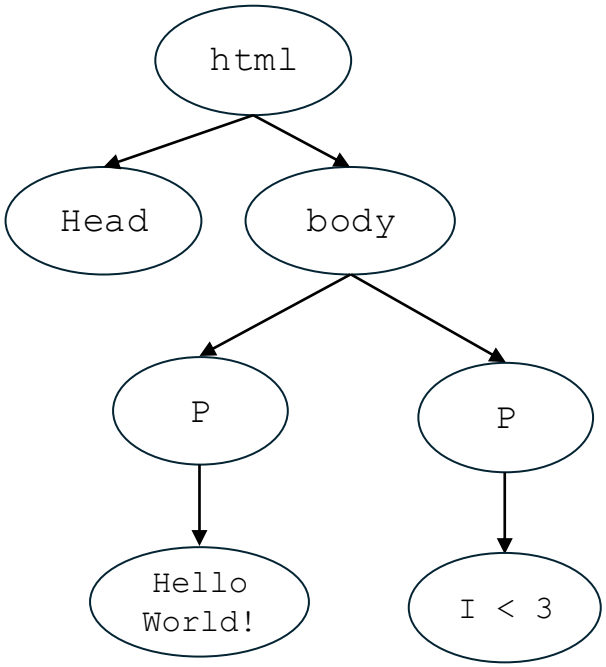
Agenda

- Review
- Client-side dynamic pages
- JavaScript
- Event-driven programming
- Summary

Review: Document Object Model (DOM)

- The DOM is a data structure built from the HTML
- HTML tags form a tree

```
<html>
  <head></head>
  <body>
    <p>Hello world!</p>
    <p>I &lt;3 chickens</p>
  </body>
</html>
```



Review: server-side dynamic pages

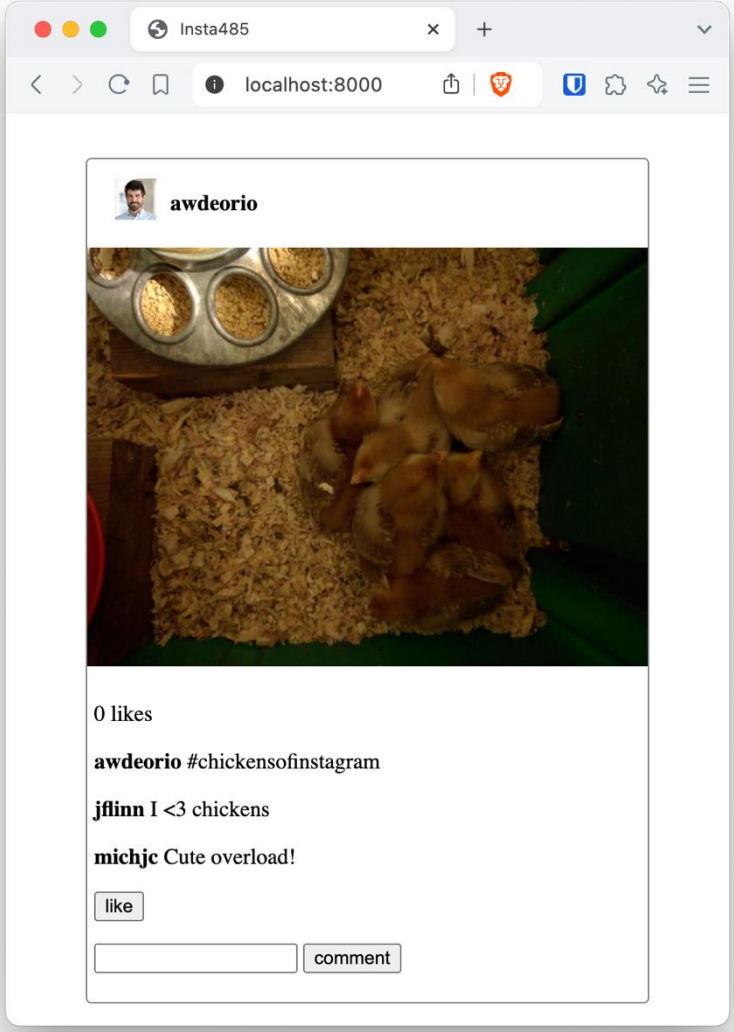
- Server-side dynamic pages: Response is the output of a function.

- Client makes a request
- Server executes a function
  - Output is usually HTML
- Server response is the output of the function

Server-side dynamic pages require reload

- With server-side dynamic pages, if one part of the page updates, the entire page must reload

- Example: Project 2 like button



Server-side dynamic pages limitations

- More limitations of server-side dynamic pages
  - No in-browser chat
  - No browser-based field validation
    - E.g., password strength, valid email address format
  - No grabbable maps
  - No deferred data loading in Gmail
  - Add/delete comment without a page reload
  - Like/unlike without a page reload
  - Double click to like
  - Infinite scroll

Agenda

- Review
- **Client-side dynamic pages**
- JavaScript
- Event-driven programming

Client-side dynamic pages

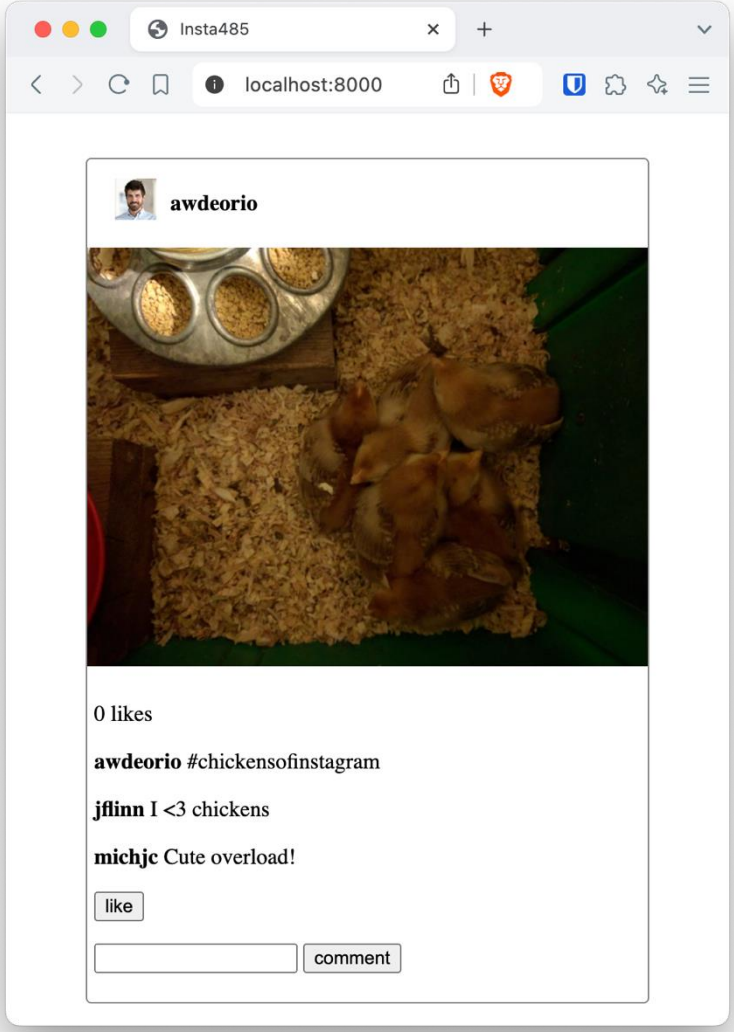
- Problem: Server code can't change the page after it has been generated and transferred to the client’s browser

- Solution: Client-side dynamic pages - Code running in the client’s browser modifies the DOM. The rendered page changes.

Client-side dynamic pages *no reload*

- With client-side dynamic pages, you can update just one part of the page

- Example: Project 3 like button



Client-side dynamic pages use JavaScript

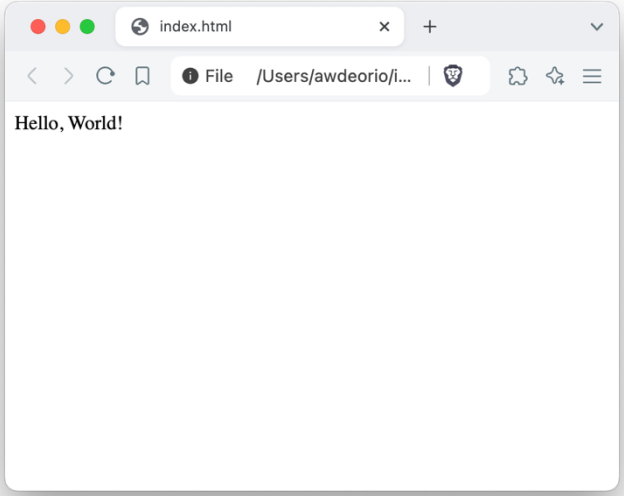
- We need a programming language in the client's web browser
- We need functions to manipulate the DOM

"JavaScript has sharp edges, but we're going to use it safely."

Agenda

- Review
- Client-side dynamic pages
- **JavaScript**
- Event-driven programming

`document` API



- The `document` API represents the web page
  - Document Object Model (DOM)
- The `document` API modifies the DOM
  - When the DOM changes, the rendered page changes

```
<html>
  <body>
    <script>
     document.write("Hello, World!");
    </script>
  </body>
</html>
```

`console` API

- `console` API writes text to the developer console

```
<html>
  <body>
    <script>
     console.log("Hello, World!");
    </script>
  </body>
</html>
```

What is JavaScript?

- "Python with C++ syntax"
- With more design flaws
- Only programming language that web browsers support
- Has nothing to do with Java

Why JavaScript?

- It's the only programming language browsers understand
- Specialty is *event-driven* and *asynchronous* applications

JavaScript reputation

- Syntax pitfalls
  - `==` vs. `===`
  - `x=5;` (global) vs. `let x=5;` (local)
  - `var` vs. `let` and `const`

- Funny-looking loops

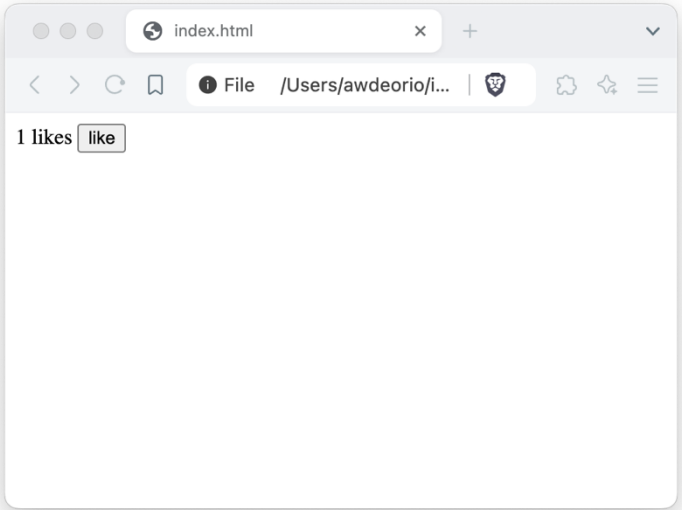
```
const chickens = ['Magda', 'Marilyn', 'Myrtle II'];
chickens.forEach((chicken) => {
  console.log(chicken);
});
```

- Confusion about *event-driven* and *asynchronous* programming
  - Covered in lecture

Example: like button

- Use the `document` API to change the DOM

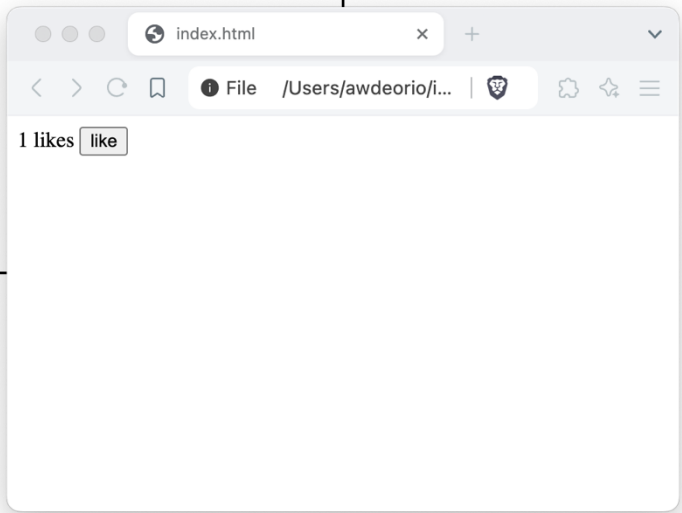
```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
</body></html>
```



Example: like button

- Use the `document` API to change the DOM

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0; // HACK global
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   addLike();
  </script>
</body></html>
```



JS changes DOM -> rendered page changes

- JavaScript programs change the DOM
- When the DOM changes, the rendered page changes
  - No page reload
  - No server interaction required
- All this happens in the client's browser
- This is client-side dynamic pages

Agenda

- Review
- Client-side dynamic pages
- JavaScript
- **Event-driven programming**

Event-driven programming

- Problem: How to run `addLike()` on button press?
  - More generally, how to respond to user input?
- Solution: Event-driven programming

- In event-driven programming, the flow of the program is determined by external events like user input.

- Built-in support from the JavaScript interpreter and the browser

Not event-driven programming

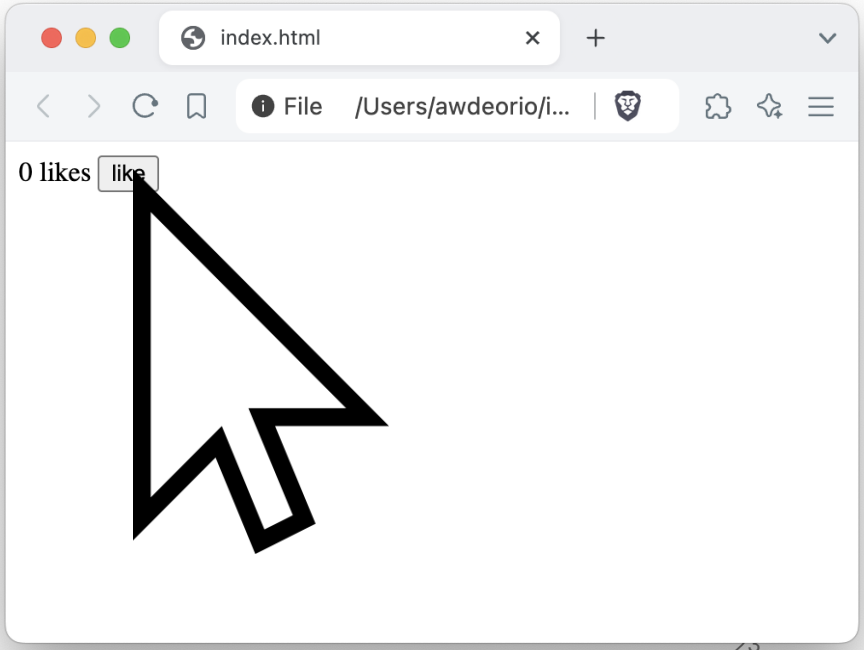
- Our previous version specified everything the program did
  - It did not respond to user input
- Next, we will let the user determine what the program does

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0; // HACK global
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   addLike();
  </script>
</body></html>
```

Events

- An *event* is an action from outside handled by the program
- *Events* often come from outside the program
  - User clicks a button
  - User scrolls to the end of the page
  - Network response arrives

- Full list of events



Events

- A button click is an *event*
- JavaScript *click*

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0; // HACK global
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById('likeBtn');
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

Event handler

- An *event handler* is a function
- An *event handler* runs after an *event* occurs
- AKA callback

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0; // HACK global
   function addLike() {
     numLikes += 1;
     const n = document.getElementById(numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

Event table

Event Table	
likeBtn **click**	**addLike**

- The *event table* maps *events* to *event handlers*
  - Nomenclature is not universal
- **`addEventListener()`** updates the event table

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0; // HACK global
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById('likeBtn');
   btn.addEventListener('click', addLike);
  </script>
</body></html>
```

Event Queue	head
`addLike`	

Event queue

- Problem: The interpreter is busy when an event occurs
- Solution: Add *event handlers* to the *event queue*

- The *event queue* stores *event handlers* waiting to execute

Stack

- Problem: Event handlers need to be executed
- Solution: Stack

- The *stack* executes functions
  - Same as 100-level programming
- When the *stack* is empty, an *event handler* is removed from the *event queue* and executed

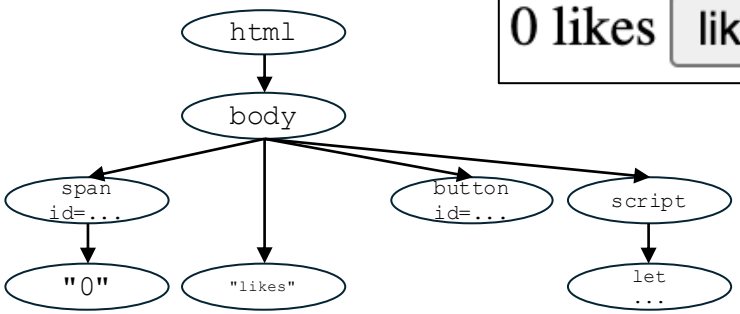
Event loop

- Problem: How to check for events, enqueue, and execute them?
- Solution: The *event loop*
- The *event loop*
  - Checks for events
  - Adds *event handlers* to the *event queue*
  - Executes a function from the *event queue*
- The *event loop* is built into the interpreter

```
while (True) {
  handler = check_event_table();
  event_queue.push(handler);
  if (!event_queue.empty()) {
    fxn = event_queue.pop();
    fxn();  // execute
  }
}
```

One button click

DOM



Rendered Page

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById('likeBtn');
   btn.addEventListener('click', addLike);
  </script>
</body></html>
```

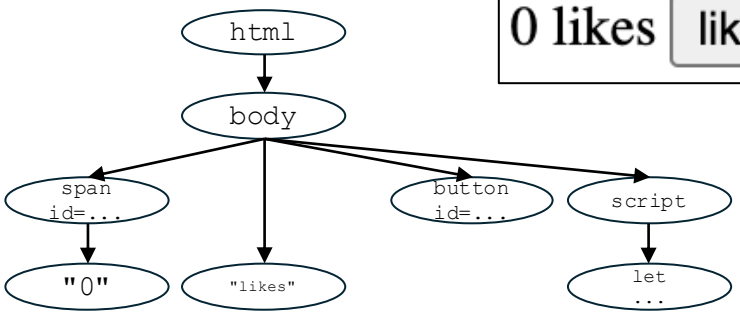
Event Table	
likeBtn **click**	**addLike**

Event Queue head

Stack

One button click

DOM



Rendered Page

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById(numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

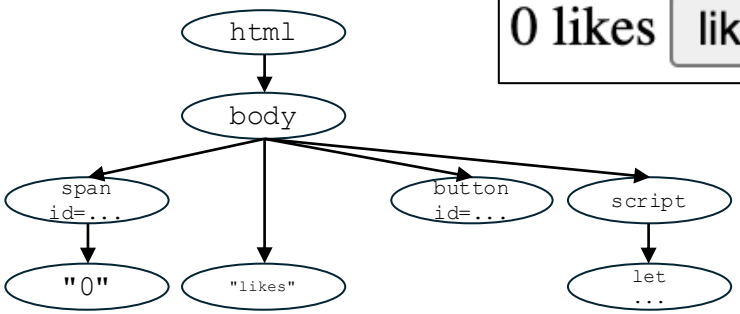
Event Table	
likeBtn **click**	**addLike**

Event Queue	head
addLike()	

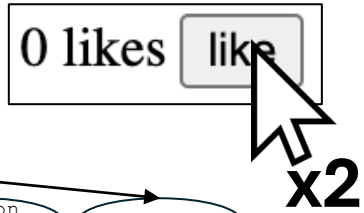
Stack

Two fast clicks

DOM



Rendered Page



x2

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById('likeBtn');
   btn.addEventListener('click', addLike);
  </script>
</body></html>
```

Event Table	
likeBtn **click**	**addLike**

Event Queue	head

Stack

JavaScript is single-threaded

- JavaScript is single-threaded
- It can only do one thing at a time

Long-running function

- What happens to our example if there's a long-running function?

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById(numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
   long_running_function();
  </script>
</body></html>
```

Long-running function

- What happens to our example if there's a long-running function?
- Everything else must wait
- The like button doesn't work while the function runs

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById('likeBtn');
   btn.addEventListener('click', addLike);
   long_running_function();
  </script>
</body></html>
```

Multiple events

- What happens to our example if there are two events?

```
<html>
  <body>
    <span id="numLikes">0</span> likes
    <button id="likeBtn">like</button>

    <script>
     let numLikes = 0; // HACK global
     function addLike() {
       numLikes += 1;
       const n = document.getElementById('numLikes');
       n.textContent = numLikes;
     }
     function subLike() {
       numLikes -= 1;
       const n = document.getElementById('numLikes');
       n.textContent = numLikes;
     }
     const btn = document.getElementById('likeBtn";
     btn.addEventListener('mouseenter',addLike);
     btn.addEventListener('mouseleave', subLike);
    </script>
  </body>
</html>
```

Multiple events

- What happens to our example if there are two events?
- Two entries in the event table

```
<html>
  <body>
    <span id="numLikes">0</span> likes
    <button id="likeBtn">like</button>

    <script>
     let numLikes = 0; // HACK global
     function addLike() {
       numLikes += 1;
       const n = document.getElementById('numLikes');
       n.textContent = numLikes;
     }
     function subLike() {
       numLikes -= 1;
       const n = document.getElementById('numLikes');
       n.textContent = numLikes;
     }
     const btn = document.getElementById("likeBtn");
     btn.addEventListener("mouseenter", addLike);
     btn.addEventListener("mouseleave", subLike);
    </script>
  </body>
</html>
```

Event Table	
likeBtn **mouseenter**	**addLike**
likeBtn **mouseleave**	**subLike**

Summary

- Client-side dynamic pages: Code running in the client's web browser modifies the DOM. The rendered page changes.
- In *event-driven programming*, the flow of the program is determined by external events like user input.
- The *event loop* checks for *events* and dispatches *event handlers* to the *event queue*
- When the *stack* is empty, an *event handler* is removed from the *event queue* and executed